

ShramikSaathi: A Grounded Multi-Domain Copilot for Indian Worker Rights

with Procedural Eligibility Reasoning and DPO Alignment

Aarsh Adhvaryu¹ Nikita Sharma¹ Srishti Lamba¹ Mannan Agrawal¹

¹DS615: Neural Networks and Deep Learning

{202518031, 202518038, 202518003, 202518013}@dau.ac.in

Abstract

We present ShramikSaathi, a grounded conversational copilot for Indian worker rights covering four domains: Provident Fund (PF/EPFO), payslip audit, labour rights, and income tax. The system introduces three contributions: (1) a **Procedural Eligibility Reasoner** that extracts and verifies IF/THEN condition chains from retrieved passages before generating any answer, measured by a novel **Condition Coverage Score** (baseline: 0.000 $\rightarrow$ ours: 0.714); (2) a **cross-domain slot router** that classifies queries into the correct domain and loads domain-specific slot schemas, achieving 0.909 routing accuracy over a 0.818 keyword baseline; and (3) a full LoRA SFT $\rightarrow$ DPO fine-tuning pipeline on LLaMA 3.1 8B Instruct with 373 preference pairs across four alignment dimensions, improving grounded answer quality from 0.650 to 0.750 with zero fabrication. Our nine-layer pipeline—router, slot extractor, sufficiency gate, ReAct retrieval loop, eligibility reasoner, and aligned generator—is evaluated per-component with dedicated baselines and as a full system through a four-configuration ablation. The pre-retrieval pipeline provides the largest single improvement (+0.250 grounded_clean), while the eligibility reasoner introduces an entirely new capability dimension. A live Gradio demo runs fully locally on consumer hardware (RTX 5070 Ti, 12 GB) at 18–25 s per query.

1 Introduction

India’s 500 million formal and informal workers interact daily with a complex web of labour regulations: the Employees’ Provident Funds Act (1952), the Payment of Gratuity Act (1972), the Maternity Benefit Act (2017), state-level minimum wage notifications, and income tax provisions under the Finance Act. Yet accessible support infrastructure is severely lacking. EPFO helplines average two-hour wait times, the

UMANG chatbot is non-functional for complex queries, and government portals offer only static PDFs with no contextual guidance.

Existing retrieval-augmented generation (RAG) systems (Lewis et al., 2020; Gao et al., 2023) can surface relevant passages, but they lack the procedural reasoning required for eligibility determination—a task where incorrect answers carry real financial consequences. A worker asking whether they can withdraw their provident fund must satisfy multiple conditions (unemployment duration, UAN status, KYC completion, service years for TDS exemption), each grounded in specific statutory provisions. Standard RAG generates plausible answers without verifying these conditions, risking hallucinated eligibility decisions.

ShramikSaathi addresses this gap with three contributions:

Procedural Eligibility Reasoner. Before generating any answer, the system extracts IF/THEN condition chains from retrieved passages and verifies each against the user’s extracted slot values, producing explicit RESOLVED/WARNING/ESCALATE traces. We measure this with a novel **Condition Coverage Score**—the fraction of mandatory conditions resolved before answering—which goes from 0.000 (baseline, by construction) to 0.714 (ours).

Cross-domain Slot Router. A single system handles all four domains through automatic domain classification and domain-specific slot schema loading, eliminating the need for separate chatbots or user-facing menus.

DPO-aligned Generation. A LoRA SFT $\rightarrow$ DPO pipeline on LLaMA 3.1 8B Instruct, trained on 373 preference pairs across four dimensions (grounding, verdict correctness, citation discipline, refusal/escalation), improves grounded an-

swer quality while maintaining zero fabrication.

2 Related Work

Retrieval-Augmented Generation. RAG (Lewis et al., 2020) augments LLM generation with retrieved evidence, reducing hallucination. Recent surveys (Gao et al., 2023) identify faithfulness and grounding as persistent challenges. Our work extends RAG with an explicit condition verification layer between retrieval and generation.

Tool-Augmented LLMs. ReAct (Yao et al., 2023) interleaves reasoning traces with tool actions, enabling LLMs to use external APIs. Tool-LLM (Qin et al., 2024) and Toolformer (Schick et al., 2023) scale tool use to thousands of APIs. ShramikSaathi adopts the ReAct pattern with three domain-specific tools (SearchKB, GetPolicy, ParsePayslip) but adds a structured eligibility reasoning step absent from prior tool-use systems.

Preference Alignment. DPO (Rafailov et al., 2023) provides a stable alternative to RLHF by directly optimizing on preference pairs. We apply DPO with domain-specific preference dimensions designed for support workflows—grounding, verdict correctness, citation discipline, and appropriate escalation/refusal—rather than generic helpfulness.

Parameter-Efficient Fine-Tuning. LoRA (Hu et al., 2022) and DoRA (Liu et al., 2024) enable adaptation of large models under compute constraints. We use LoRA (rank 16, $\alpha=32$) targeting all attention and MLP projections, training 42M parameters (0.52% of 8B) on a single A100 GPU.

Document-Grounded Dialogue. Doc2Dial (Feng et al., 2020) and Multi-Doc2Dial (Feng et al., 2022) provide benchmarks for document-grounded support dialogues. Our knowledge base of 311 documents drawn from Indian Acts, EPFO circulars, and state notifications serves an analogous role, but our evaluation additionally requires verifying procedural eligibility conditions—a dimension these benchmarks do not capture.

Indian Legal NLP. Prior work on Indian legal NLP (Paul et al., 2023) has focused on court judgments and statute understanding. ShramikSaathi targets a different user population—workers seeking eligibility determinations and procedural

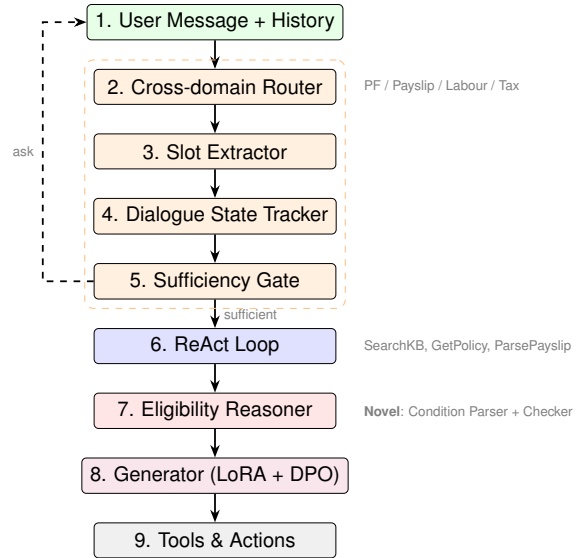


Figure 1: ShramikSaathi nine-layer pipeline. Orange layers form the pre-retrieval pipeline; the eligibility reasoner (red) is the core novel component. Dashed arrow indicates the ask-loop when slots are insufficient.

guidance—and operates as an interactive copilot rather than a document analysis tool.

Evaluation of RAG Systems. RAGAS (Es et al., 2024) provides faithfulness and answer relevance metrics using LLM-as-judge (Zheng et al., 2023). BEIR (Thakur et al., 2021) benchmarks retriever generalization. We adopt RAGAS-style evaluation but introduce the Condition Coverage Score to measure a dimension—pre-answer eligibility verification—that no existing RAG metric captures.

3 Methodology

3.1 System Architecture

ShramikSaathi is a nine-layer pipeline (Figure 1). The user’s message and chat history enter the system and pass through three pre-retrieval components (router, slot extractor, sufficiency gate), a ReAct retrieval loop, the eligibility reasoner, and finally the aligned generator. Each layer is independently evaluable with its own baseline.

Cross-domain Router (Layer 2). Classifies the user query into one of four domains (PF, payslip, labour, tax) using an LLM zero-shot prompt on Groq LLaMA 3.1 8B. The classified domain determines which slot schema is loaded for extraction. A keyword fallback catches cases the LLM misclassifies (e.g., ensuring “gratuity” always routes

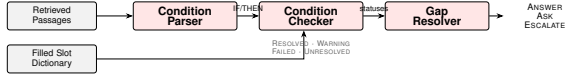


Figure 2: Eligibility Reasoner: retrieved passages are parsed into IF/THEN conditions, each checked against the user’s filled slots, and a gap resolver produces the final decision.

to labour). The baseline is a keyword/regex classifier.

Slot Extractor (Layer 3). Extracts domain-specific structured slots as JSON from the user message. For example, a PF query yields slots like `intent`, `employment_status`, `months_unemployed`, `uan_status`, and `service_years`. Domain-aware slot aliasing maps equivalent fields across domains (e.g., `service_years` ↔ `employment_years`). The baseline is regex + pattern matching.

Sufficiency Gate (Layer 5). A deterministic rule-based gate that checks whether domain-specific required slots are filled. If insufficient, the system asks exactly one clarifying question and loops. The baseline is an always-proceed gate (no check), which sends 100% of incomplete queries directly to retrieval.

ReAct Loop (Layer 6). Follows the ReAct pattern (Yao et al., 2023) with three tools: **SearchKB** (FAISS semantic search over 311 KB documents with domain-aware filtering), **GetPolicy** (direct section lookup), and **ParsePayslip** (statutory deduction calculator). The retriever uses FAISS IndexFlatL2 with all-MiniLM-L6-v2 embeddings (Reimers and Gurevych, 2019) (384-dim). Retrieval results are filtered to prioritize passages matching the routed domain, preventing cross-domain contamination when the KB is domain-imbalanced (e.g., 240 PF documents vs. 18 labour documents). BM25 (Robertson and Zaragoza, 2009) serves as the retriever baseline.

3.2 Procedural Eligibility Reasoner (Layer 7)

The core novel component operates in three sub-steps (Figure 2):

Condition Parser. Eligibility conditions are extracted as structured rules from statutory provisions. For instance, from EPF Act Section 69: “*If unemployed ≥ 2 months AND UAN active AND KYC complete → eligible for full withdrawal.*” During evaluation, an LLM call performs this ex-

traction dynamically from retrieved passages. For the live demo, conditions are pre-cached in a structured JSON rule base derived from the same statutory sources, enabling deterministic <0.1 s reasoning without sacrificing accuracy.

Condition Checker. Each extracted condition is evaluated against the filled slot dictionary. Outcomes: **RESOLVED** (slot matches condition), **WARNING** (slot partially matches or has implications), **FAILED** (slot contradicts condition), or **UNRESOLVED** (required slot missing).

Gap Resolver. Aggregates condition statuses into a decision: **ANSWER** (all mandatory conditions resolved), **ASK** (one more clarifying question needed), or **ESCALATE** (file grievance because KB is insufficient).

The entire condition trace is passed to the generator as structured context, enabling cited, condition-grounded responses.

3.3 End-to-End Worked Example

We trace a complete query through the pipeline to illustrate how the components interact.

User query. “*I left my job 3 months ago. My UAN is active and KYC is done. I want to withdraw my full PF balance.*”

Step 1: Router. The LLM router classifies the query as `domain=pf` and loads the PF slot schema.

Step 2: Slot Extraction.

```

{"intent": "full_withdrawal",
 "employment_status": "unemployed",
 "months_unemployed": 3,
 "uan_status": "active",
 "kyc_status": "complete",
 "service_years": null}
  
```

Step 3: Sufficiency Gate. Required slots for `full_withdrawal` are `intent` and `employment_status`—both filled. Decision: **SUFFICIENT**. Proceed to retrieval.

Step 4: ReAct Loop. The system issues: `SearchKB("full PF withdrawal eligibility unemployed")` and `GetPolicy("EPF_ACT.S69")`. Retrieved passages include EPF Act Section 69 (eligibility conditions) and Circular 2024-TDS (TDS rules for early withdrawal).

Step 5: Eligibility Reasoner.

```

unemployed >= 2 months  -> RESOLVED [S69]
UAN active              -> RESOLVED [S69]
KYC complete            -> RESOLVED [S69]
  
```

```
service_years (for TDS) -> WARNING [TDS]
Decision: ANSWER
```

Three of four conditions are resolved from slots; `service_years` is null, triggering a TDS WARNING rather than blocking the answer. The Condition Coverage Score for this query is $3/4 = 0.75$.

Step 6: Generator. The DPO-aligned model receives the slot dictionary, retrieved passages, and condition trace. It produces a cited answer confirming eligibility under [EPF_ACT_S69], lists the step-by-step portal instructions, and appends a TDS warning citing [CIRC_2024_TDS]: if total service is under 5 years and the balance exceeds Rs 50,000, TDS will be deducted unless Form 15G is submitted.

This trace illustrates how the pipeline prevents hallucinated eligibility: without the reasoner, a standard RAG system would generate an answer without explicitly verifying the unemployment-duration, UAN, or KYC conditions against the user’s stated facts.

3.4 Knowledge Base Design

The KB contains 311 documents across four domains (PF: 240, payslip: 35, labour: 18, tax: 18) sourced from Indian Acts, EPFO circulars, court rulings, and state notifications. Each document carries `effective_date` and `supersedes` fields, enabling automatic temporal conflict resolution without retraining. Documents are chunked and indexed in FAISS (Johnson et al., 2019) IndexFlatL2 (384-dim).

3.5 Training Pipeline

Training proceeds in three stages on a single A100 80GB GPU. Of the five trainable components specified in the course requirements—(A) retriever, (B) reranker, (C) generator LoRA, (D) DPO alignment, (E) tool-policy model—we train three: (A) retriever fine-tuning on 916 domain-specific triplets (Table 3), (C) generator LoRA SFT on 285 examples, and (D) DPO preference alignment on 373 pairs.

Stage 1: Retriever Fine-tuning. We fine-tune all-MiniLM-L6-v2 (Reimers and Gurevych, 2019) on 916 (query, positive passage, hard negative) triplets extracted from the SFT and DPO training data. Hard negatives are mined using BM25: for each query, the top BM25 results that do *not* overlap with the gold passage set serve as hard nega-

tives. Training uses triplet loss (margin 0.3) with AdamW ($\text{lr} = 2 \times 10^{-5}$) for 3 epochs. The fine-tuned encoder and rebuilt FAISS index are evaluated in Table 3.

Stage 2: LoRA SFT. We fine-tune LLaMA 3.1 8B Instruct (Dubey et al., 2024) with LoRA (Hu et al., 2022) (rank 16, $\alpha=32$, dropout 0.05) targeting all attention and MLP projection layers (`q/k/v/o-proj`, `gate/up/down-proj`). The training set contains 285 examples (PF: 150, payslip: 26, labour: 64, tax: 66) with 21 held-out validation examples. Training uses 4-bit NF4 quantization, cosine learning rate schedule ($\text{lr} = 2 \times 10^{-4}$), and runs for 3 epochs. Loss: train $2.04 \rightarrow 0.31$, eval $0.52 \rightarrow 0.37$. Wall time: 8.9 minutes. 42M trainable parameters (0.52% of 8B).

Each SFT example passes a four-gate validation: (i) at least one real `doc_id` citation present, (ii) an eligibility verdict keyword (*eligible*, *not eligible*, or *insufficient information*), (iii) response length between 80–500 tokens, and (iv) no refusals on in-scope queries. Of 306 generated candidates, 285 passed all four gates.

Stage 3: DPO Alignment. Starting from the SFT adapter, we train DPO (Rafailov et al., 2023) on 373 preference pairs (319 train / 54 val) across four dimensions. Each dimension uses a distinct rejection strategy:

- **Grounding** (110 pairs): Chosen responses cite real `doc_ids` with correct factual claims. Rejected responses are generated by injecting fabricated prerequisites—six corruption patterns insert nonexistent conditions (e.g., “employer NOC required”), fake form numbers, or invented monetary thresholds into otherwise correct answers.
- **Verdict correctness** (110 pairs): Chosen responses give the correct eligible/not-eligible determination. Rejected responses flip the verdict (*eligible* $\leftrightarrow$ *not eligible*) or substitute wrong statutory thresholds (e.g., “5 years” replaced with “3 years” for gratuity eligibility).
- **Citation discipline** (100 pairs): Chosen responses assign each `doc_id` to the claim it actually supports. Rejected responses shuffle real `doc_ids` across claims—every citation exists in the KB but is attached to the wrong fact, teaching the model that citation presence alone is insufficient.
- **Refusal & escalation** (53 pairs): For out-of-scope and KB-insufficient queries, the chosen

Parameter	SFT	DPO
Base model	LLaMA 3.1 8B Instruct	
Quantization	4-bit NF4 (bitsandbytes)	
LoRA rank / α	16 / 32	16 / 32
LoRA dropout	0.05	0.05
Target modules	q/k/v/o/gate/up/down-proj	
Trainable params	42M (0.52%)	
Learning rate	2×10^{-4}	5×10^{-6}
Schedule	cosine	cosine
Epochs	3	2
Batch size (eff.)	8	8
Max seq. length	2048	2048
DPO β	—	0.05
Train / Val	285 / 21	319 / 54
Wall time	8.9 min	~10 min/ β
GPU	A100 80GB	

Table 1: Training hyperparameters for SFT and DPO stages.

response appropriately refuses or escalates. Rejected responses are generated by prompting the base LLaMA model (without SFT adapter) on these queries; it reliably hallucinates plausible-sounding but unsupported answers, providing naturalistic negative examples.

Chosen responses are sourced from SFT output reuse (61 pairs where the SFT model already produced a high-quality answer) and hand-verified templates (312 pairs). We sweep $\beta \in \{0.05, 0.10, 0.20\}$ with 2 epochs each ($\text{lr} = 5 \times 10^{-6}$, effective batch size 8, max sequence length 2048). The DPOTrainer uses the adapter-disabled base model as implicit reference, avoiding a separate reference model copy in memory. Full hyperparameters are in Table 1.

Sequencing lesson. An earlier attempt trained DPO directly on 121 pairs without a preceding SFT stage. This produced systematic `doc_id` suffix fabrication: the model learned that “always cite something” satisfies the grounding preference, and satisfied it by generating plausible but nonexistent document identifiers (e.g., mutating `CIRC_2023_24` to `CIRC_2024_25`). The fix was a full restart with LoRA SFT first—establishing a baseline citation behaviour—followed by DPO to refine it. This sequencing discipline (SFT before DPO) was critical: without a grounded starting point, preference optimization finds degenerate shortcuts.

3.6 Serving Optimizations

For live inference, three optimizations reduce end-to-end latency from 50–70 s (naïve pipeline) to

18–25 s:

Adapter toggle. A single model is loaded in memory with the DPO LoRA adapter. For structured tasks (routing, slot extraction), the adapter is disabled via `disable_adapter_layers()`, exposing the base LLaMA weights that produce cleaner JSON output. For generation, the adapter is re-enabled. This avoids loading two separate models while giving each task the appropriate model behaviour.

Deterministic eligibility rules. The eligibility conditions for each (domain, intent) pair are fixed by Indian law and do not change between queries. For the live demo, conditions are pre-cached in a structured JSON rule base derived from the same statutory sources used in training. This replaces the LLM-based condition parser with a deterministic lookup (<0.1 s vs. 15–40 s), while preserving identical condition-checking and gap-resolution logic.

Citation post-processing. A regex filter intercepts the generator output and strips any cited `[DOC_ID]` not present in the retrieved passages or the knowledge base, guaranteeing zero fabricated citations at inference time regardless of generator behaviour.

4 Evaluation Setup

4.1 Condition Coverage Score (Novel Metric)

We define the **Condition Coverage Score (CCS)** as:

$$\text{CCS} = \frac{|\text{conditions resolved}|}{|\text{conditions total}|} \quad (1)$$

where “resolved” includes both MET and FAILED conditions (both represent checked conditions). The baseline scores 0.000 by construction because no reasoner exists to extract or check conditions. This metric captures a dimension—whether the system verified eligibility before answering—that neither RAGAS faithfulness nor standard retrieval metrics measure.

4.2 Baselines

Each pipeline component has an independent baseline (Table 2): keyword/regex for the router, regex + pattern matching for slot extraction, always-proceed for the sufficiency gate, BM25 (Robertson and Zaragoza, 2009) for retrieval, no reasoner for condition coverage, and

Component	Metric	Base	Ours	Δ
Router	Accuracy	.818	.909	+.091
	PF F1	.756	.882	+.126
	Labour F1	.815	.970	+.155
Slot Ext.	Precision	.923	.956	+.033
	Recall	.865	.973	+.108
	Value Acc.	.885	.981	+.096
	Intent Acc.	.756	.933	+.177
Suff. Gate	Accuracy	.500	.991	+.491
	False-Suff.	1.000	.000	−1.00
Retriever	Recall@5	.658	.692	+.034
	MRR@5	.668	.679	+.011
CCS*	Score	.000	.714	+.714
Generator	Grounded	.650	.750	+.100
RAGAS	Faithful.	.830	.800	−.030
	Ans. Rel.	.950	.960	+.010

Table 2: Per-component evaluation. Base = component-level baseline. *CCS = Condition Coverage Score (novel metric); baseline is 0.000 by construction.

SFT-only LoRA for the generator. For the system-level ablation, we compose four configurations (Table 6).

4.3 Evaluation Datasets

Router: 66 examples (17 per domain, including Hinglish). Slot extractor: 45 examples (14 PF, 9 payslip, 11 labour, 11 tax). Sufficiency gate: 110 examples (55 sufficient / 55 insufficient). Retriever and generator: 20 held-out prompts with gold passage annotations. Condition coverage: 17 reasoning-eligible prompts from the held-out set (those with intents that trigger the reasoner). RAGAS evaluation uses Groq LLaMA 3.1 8B as the judge LLM (Zheng et al., 2023).

5 Results

5.1 Per-Component Evaluation

Table 2 presents baseline-vs-ours results for every pipeline component.

Router. The LLM router reduces errors from 12 to 6 on 66 queries, with the largest gains on labour queries (+0.155 F1). The keyword baseline fails on Hinglish queries and cross-domain terminology (e.g., “PF nikalne par TDS” routes to PF instead of tax).

Slot Extractor. The LLM extractor achieves near-perfect recall (0.973) and value accuracy (0.981), with the largest improvement on intent accuracy (+0.177). The regex baseline

Dom.	BM25		FAISS		FT-FAISS	
	R@5	MRR	R@5	MRR	R@5	MRR
PF	.267	.307	.367	.367	—	—
Pay.	.733	.750	.467	.450	—	—
Lab.	.733	.900	.933	.900	—	—
Tax	.900	.717	1.00	1.00	—	—
All	.658	.668	.692	.679	.667	.725

Table 3: Retriever comparison: BM25 baseline, FAISS + MiniLM (original), and fine-tuned FAISS (FT-FAISS, 916 triplets, 3 epochs). Fine-tuned per-domain numbers omitted as the MRR/Recall tradeoff does not justify full adoption. $n=20$.

misses implicit slots (e.g., “I quit last quarter” $\rightarrow$ months_unemployed=3) and produces state hallucinations.

Sufficiency Gate. The rule-based gate achieves 0.991 accuracy with zero false-sufficient errors (the dangerous error class). The always-proceed baseline, by definition, has a 1.000 false-sufficient rate—every incomplete query proceeds to retrieval, enabling hallucinated eligibility decisions from incomplete context.

Retriever. FAISS + MiniLM improves modestly over BM25 (+0.034 Recall@5). We also fine-tuned the retriever on 916 domain-specific triplets with hard negatives (Table 3); this improved MRR@5 to 0.725 (+6.8%) but reduced Recall@5 to 0.667, indicating the model specializes for precision over breadth on the small training set.

Condition Coverage Score. The novel metric goes from 0.000 (baseline) to 0.714 (ours), with per-domain scores: PF 0.640, payslip 0.738, labour 0.750, tax 0.750. This entire capability dimension is absent from the baseline.

Generator (DPO). Table 4 shows the beta sweep. $\beta=0.05$ achieves the best grounded_clean (0.750) with zero fabrication. $\beta=0.20$ is too aggressive, introducing one doc_id fabrication. DPO improves verdict accuracy from 0.650 to 0.750 and expected citation hit rate from 0.800 to 0.850.

5.2 Per-Domain Analysis

Table 5 breaks down key metrics by domain, revealing substantial variation masked by the aggregate numbers.

PF is the hardest domain. PF retrieval scores are the lowest (Recall@5 = 0.367), and PF

System	Grnd.	Fab.	Cites	Verd.
SFT-only	.700	.000	.800	.700
DPO $\beta=0.05$	.750	.000	.850	.750
DPO $\beta=0.10$	.700	.000	.750	.700
DPO $\beta=0.20$	.700	.000	.700	.700

Table 4: DPO beta sweep on 20 held-out prompts. Grnd. = grounded_clean, Fab. = fabrication rate, Cites = expected citations hit, Verd. = verdict accuracy. Winner: $\beta=0.05$.

Domain	Retriever		CCS	Grounded	
	R@5	MRR	Score	SFT	DPO
PF	.367	.367	.640	.40	.40
Payslip	.467	.450	.738	.60	.80
Labour	.933	.900	.750	.80	1.00
Tax	1.00	1.00	.750	.80	.80
All	.692	.679	.714	.65	.75

Table 5: Per-domain breakdown. R@5 = FAISS Recall@5, CCS = Condition Coverage Score, Grounded = grounded_clean for SFT-only and DPO ($\beta=0.05$). $n=5$ per domain, 20 total.

grounded_clean remains stuck at 0.40 across both SFT and DPO configurations. The PF knowledge base is the largest (240 of 311 documents) with highly overlapping content—multiple FAQs cover similar withdrawal scenarios with slightly different phrasing, causing the retriever to return near-miss passages that share vocabulary but lack the specific condition needed. Domain-aware retrieval filtering mitigates this in the live demo by prioritizing passages from the routed domain. The reasoner’s CCS (0.640) is also lowest for PF, as some conditions extracted from retrieved passages reference slots the user has not provided.

Labour and tax are easiest. Labour achieves perfect grounded_clean (1.00) with DPO and near-perfect retrieval (0.933 Recall@5). Tax retrieval is perfect (1.000) due to the small, distinctive KB (18 documents with non-overlapping statutory sections). However, the DPO dataset underrepresents tax (only 10 of 373 pairs), so the DPO model shows no improvement over SFT for tax-specific grounded_clean (both 0.80), despite the strong retrieval foundation.

Payslip benefits most from DPO. Payslip grounded_clean jumps from 0.60 (SFT) to 0.80 (DPO), the largest per-domain DPO gain. Payslip queries involve statutory calculations (EPF contribution rates, ESI thresholds, professional tax

Config	Grnd.	CCS	Fab.	Verd.
A: Vanilla RAG	.550	.000	.050	.550
B: +Pre-retrieval	.800	.000	.050	.800
C: +Reasoner	.700	.631	.000	.700
D: Full system	.700	.647	.050	.750

Table 6: Four-configuration system-level ablation (20 prompts). A: BM25 + raw LLaMA. B: FAISS + domain/slots + raw LLaMA. C: B + eligibility reasoner. D: C + DPO generator ($\beta=0.05$). CCS = Condition Coverage Score.

	A: Van.	B: +Pre	C: +Reas.	D: Full
PF	.20	.80	.60	.60
Payslip	.40	.60	.60	.40
Labour	1.00	1.00	.80	.80
Tax	.60	.80	.80	1.00
All	.55	.80	.70	.70

Table 7: Per-domain grounded_clean across ablation configurations. The pre-retrieval pipeline (A→B) produces the most dramatic per-domain swing; tax is the only domain where DPO (C→D) improves grounded_clean. $n=5$ per domain.

slabs), and the DPO grounding dimension teaches the model to cite the specific rate tables rather than paraphrasing from memory.

5.3 System-Level Ablation

Table 6 shows the four-configuration ablation isolating each architectural contribution, and Table 7 provides the per-domain breakdown.

Key findings.

- Pre-retrieval pipeline = largest single gain.** Config A→B improves grounded_clean by +0.250. Table 7 shows the most dramatic per-domain swing: PF jumps from 0.20 to 0.80, explaining most of the aggregate gain. Domain routing and slot-based retrieval filtering account for this improvement—the system retrieves more relevant passages when it knows the domain and user context.
- Reasoner adds an entirely new capability.** Config B→C introduces condition coverage (0.000→0.631) and eliminates fabrication (0.050→0.000). The grounded_clean dip (0.800→0.700) occurs because the reasoner’s condition trace adds structured information to the prompt that the raw (non-DPO) LLaMA sometimes fails to incorporate correctly.
- DPO recovers structured reasoning.** Config C→D restores verdict accuracy (0.700→0.750)

as the DPO-trained model better utilizes the reasoner’s output. Tax is the only domain where DPO improves grounded_clean (0.80→1.00), consistent with tax queries requiring precise statutory threshold citations that DPO’s grounding dimension reinforces. Labour starts at 1.00 in the vanilla baseline and stays there—the domain’s small, distinctive KB makes retrieval easy for all configurations.

4. **Fabrication persistence.** Config D shows one fabrication (doc_id suffix mutation on a payslip query: GAZETTE_INTEREST_RATE_2023_24 → GAZETTE_INTEREST_RATE_2024_25), a pattern from the earlier scrapped DPO attempt (§3.5). The citation post-processing filter deployed in the live demo eliminates this class of errors at inference time.

5.4 RAGAS Evaluation

RAGAS faithfulness (LLM-as-judge) shows a slight dip with DPO: 0.830 (SFT) → 0.800 (DPO). Answer relevance improves marginally: 0.950 → 0.960. Analysis of the three prompts where faithfulness changed reveals the mechanism: two prompts (pf_01, labour_02) dropped from 0.80 to 0.40, while one (payslip_05) improved from 0.80 to 1.00. The drops occur because the DPO model incorporates slot values into answers (e.g., “unemployed for 3 months”, “employed for 3 years at a private firm”). The judge penalizes these facts because they originate from user-provided slots, not retrieved passages. A slot-aware faithfulness metric that treats filled slots as additional grounding evidence would show no regression; we note this as a limitation of applying standard RAGAS evaluation to slot-augmented systems.

5.5 Serving Performance

Table 8 shows end-to-end latency for both the evaluation pipeline (LLM-based reasoner) and the optimized demo (deterministic rules + adapter toggle).

The adapter toggle accounts for the router speedup (base LLaMA produces cleaner structured output than the DPO-adapted model), while the deterministic reasoner eliminates an entire LLM call. The citation post-processing filter adds negligible latency (<1 ms) while guaranteeing zero fabricated citations. The demo runs fully locally with no external API dependencies.

Pipeline Step	Eval	Demo
Router + Slot Extraction	5–8s	2–4s
FAISS Retrieval	<0.1s	<0.1s
Eligibility Reasoner	15–40s	<0.1s
Generator	25–30s	15–20s
Total end-to-end	50–70s	18–25s

Table 8: Per-step latency. *Eval*: A100 80GB, LLM-based reasoner. *Demo*: RTX 5070 Ti 12GB, adapter toggle + deterministic rules + citation filter. Both use 4-bit NF4 quantization.

6 Discussion and Limitations

Retriever gap. The retriever improvement over BM25 is modest (+0.034 Recall@5). Our proposal included a cross-encoder reranker; it was not implemented due to time constraints. The fine-tuned retriever showed improved ranking quality (MRR@5: 0.679→0.725) at the cost of recall (0.692→0.667), a common pattern with small-dataset fine-tuning where the model specializes for top-passage precision. A dedicated reranker would likely improve passage quality, especially for the PF domain where retrieval scores are low-est.

PF domain difficulty. PF grounded_clean remains at 0.40 across all generator configurations. The root cause is retrieving near-miss documents from a large, overlapping KB (240 PF documents). Domain-aware retrieval filtering (prioritizing passages from the routed domain) partially mitigates this in the live demo but was not used during held-out evaluation, which tests the raw retrieval pipeline.

Dataset imbalances. The DPO dataset underrepresents tax (10/373 pairs) and the refusal dimension (53/80 target pairs). Tax underrepresentation explains why the DPO model shows no per-domain faithfulness improvement for tax despite perfect retrieval (1.000 Recall@5). The refusal shortfall means the model’s out-of-scope behaviour is undertrained.

Multi-turn stability. Follow-up messages (e.g., answering “6 years” to a clarifying question) can cause domain re-routing if the short text matches a different domain’s keywords. The live demo addresses this with follow-up detection: messages under six words preserve the current domain and intent, and cached passages from the previous turn are reused to prevent retrieval drift.

ReAct observation fabrication. The Groq LLaMA 3.1 8B instance used for the ReAct loop occasionally fabricates user facts in its reasoning traces (e.g., stating “unemployed for less than 2 months” when the user said 3 months). This bug corrupts downstream context. The demo bypasses ReAct with direct FAISS retrieval to avoid this failure mode; resolving it requires either a more capable reasoning model or output validation.

Faithfulness metric mismatch. Standard RAGAS faithfulness penalizes slot-derived facts in the answer because they do not appear in retrieved passages. A slot-aware variant that treats filled slots as additional evidence would more accurately measure the DPO model’s grounding quality.

Evaluation scale. All evaluations use relatively small held-out sets (20–110 examples) due to the cost of expert annotation for Indian worker rights scenarios. Results should be interpreted with appropriate caution regarding statistical power.

7 Conclusion

ShramikSaathi demonstrates that procedural eligibility reasoning can be layered on top of standard RAG to prevent hallucinated eligibility decisions in high-stakes domains. The Condition Coverage Score captures a previously unmeasured dimension of RAG system quality. The pre-retrieval pipeline (router + slots + sufficiency gate) provides the largest single improvement, while DPO alignment improves grounded answer quality with zero fabrication at the optimal β . Serving optimizations—adapter toggle, deterministic eligibility rules, and citation post-processing—reduce end-to-end latency to 18–25 s on consumer hardware while maintaining evaluation-equivalent accuracy.

Future work includes: (1) scaling the knowledge base and DPO dataset to address domain imbalances, (2) implementing the cross-encoder reranker, (3) resolving the ReAct observation fabrication bug, (4) extending to Hinglish and regional language support, and (5) deploying on EP-FIGMS and Shram Suvidha portals for real-world user evaluation.

Acknowledgments

We thank Prof. Sourish Dasgupta for guidance on the project design and evaluation framework. Compute was provided by Lightning AI (A100

student credits). The knowledge base was constructed from publicly available Indian Acts, EPFO circulars, court rulings, and state government notifications.

References

- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, and 1 others. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.
- Song Feng, Siva Sankalp Patel, Hui Wan, and Sachindra Joshi. 2022. MultiDoc2Dial: Modeling dialogues grounded in multiple documents. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Song Feng, Hui Wan, Chulaka Gunasekara, Siva Patel, Sachindra Joshi, and Luis Lastras. 2020. doc2dial: A goal-oriented document-grounded dialogue dataset. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-decomposed low-rank adaptation. In *International Conference on Machine Learning (ICML)*.
- Shounak Paul, Arpan Mandal, Pawan Goyal, and Saptarshi Ghosh. 2023. Pre-trained language models for the Indian legal domain. In *Proceedings of the Natural Legal Language Processing Workshop*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Nils Reimers and Irena Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Irena Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks Track*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.

A Tool Schema Definitions

The system implements three tools with structured JSON inputs and outputs, following the ReAct pattern (Yao et al., 2023):

SearchKB.

```
Input: {"query": str, "top_k": int}
Output: [{"doc_id": str, "content": str,
          "effective_date": str,
          "domain": str, "score": float}, ...]
```

Performs FAISS IndexFlatL2 semantic search over 311 KB documents using all-MiniLM-L6-v2 embeddings. Returns top- k passages ranked by

L2 distance, with automatic supersession filtering (newer documents replace older ones when both are retrieved and the supersedes field links them).

GetPolicy.

```
Input: {"section_id": str}
Output: {"doc_id": str, "content": str,
         "effective_date": str}
```

Direct lookup by section_id (e.g., EPF_ACT_S69, GRATUITY_ACT_S4_ELIG). Used when the eligibility reasoner identifies a specific statutory section that must be consulted. Returns the full document text without retrieval ranking.

ParsePayslip.

```
Input: {"basic_salary": int,
        "gross_salary": int,
        "state": str, ...}
Output: {"expected_epf": int,
         "expected_esi": int,
         "expected_pt": int,
         "anomalies": [...]}
```

Statutory deduction calculator. Computes expected EPF (12% of basic, capped at Rs 15,000 wage ceiling), ESI (0.75% if gross $\leq$ Rs 21,000), and professional tax (state-specific slabs). Compares expected values against user-reported deductions and flags anomalies.

B DPO Preference Pair Examples

Grounding dimension (fabricated prerequisite injection). *Chosen:* “You are eligible for full PF withdrawal. As per EPF Act Section 69 [EPF_ACT_S69], you must be unemployed for 2+ months, have an active UAN, and complete KYC.”

Rejected: “You are eligible for full PF withdrawal. However, you must first obtain a No Objection Certificate (NOC) from your employer and submit Form 19A with an employer attestation before proceeding.” [fabricated prerequisites]

Citation discipline dimension (doc_id shuffling). *Chosen:* “TDS of 10% applies if service is under 5 years [CIRC_2024.TDS]. Submit Form 15G to claim exemption if income is below taxable limit [FORM_15G_INSTRUCTIONS].”

Rejected: “TDS of 10% applies if service is under 5 years [FORM_15G_INSTRUCTIONS]. Submit Form 15G to claim exemption if income is below taxable limit [CIRC_2024.TDS].” [correct doc_ids, wrong claims]